

```

import React, { useEffect, useRef, useCallback } from "react";
import { Info, CheckCircle, AlertTriangle, Check } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🗝 LOCAL TYPE ISOLATION GATEWAY
export interface MessageBoxProps {
  open: boolean;
  onClose: () => void;
  title: string;
  description: string;
  variant?: "info" | "success" | "warning";
}

const variantStyles = {
  info: {
    icon: Info,
    iconColor: "text-blue-500",
    bgColor: "bg-blue-500/10",
    borderColor: "border-blue-500/30",
    ringColor: "focus-visible:ring-blue-500",
  },
  success: {
    icon: CheckCircle,
    iconColor: "text-emerald-500",
    bgColor: "bg-emerald-500/10",
    borderColor: "border-emerald-500/30",
    ringColor: "focus-visible:ring-emerald-500",
  },
  warning: {
    icon: AlertTriangle,
    iconColor: "text-amber-500",
    bgColor: "bg-amber-500/10",
    borderColor: "border-amber-500/30",
    ringColor: "focus-visible:ring-amber-500",
  },
} as const;

const variantLabels = {
  info: "Information",
  success: "Success",
  warning: "Warning",
} as const;

export function MessageBox({

```

```

open,
onClose,
title,
description,
variant = "info",
}: MessageBoxProps) {
  const dialogRef = useRef<HTMLDialogElement>(null);
  const previousActiveElement = useRef<HTMLElement | null>(null);
  const {
    icon: Icon,
    iconColor,
    bgColor,
    borderColor,
    ringColor,
  } = variantStyles[variant];

  const handleKeyDown = useCallback(
    (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        event.preventDefault();
        onClose();
      }
    },
    [onClose],
  );

  const handleFocusTrap = useCallback((event: KeyboardEvent) => {
    if (event.key !== "Tab" || !dialogRef.current) return;

    const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
    );

    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];

    if (event.shiftKey && document.activeElement === firstElement) {
      event.preventDefault();
      lastElement?.focus();
    } else if (!event.shiftKey && document.activeElement === lastElement) {
      event.preventDefault();
      firstElement?.focus();
    }
  }, []);

  useEffect(() => {
    if (open) {
      previousActiveElement.current = document.activeElement as HTMLElement;
      document.addEventListener("keydown", handleKeyDown);
      document.addEventListener("keydown", handleFocusTrap);
    }
  }, [open, previousActiveElement, handleKeyDown, handleFocusTrap]);
}

```

```

document.body.style.overflow = "hidden";

dialogRef.current?.showModal();

const focusableElement = dialogRef.current?.querySelector<HTMLElement>(
  'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])',
);
focusableElement?.focus();

return () => {
  document.removeEventListener("keydown", handleKeyDown);
  document.removeEventListener("keydown", handleFocusTrap);
  document.body.style.overflow = "";
  previousActiveElement.current?.focus();
  dialogRef.current?.close();
};
}, [open, handleKeyDown, handleFocusTrap]);

if (!open) return null;

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4">
    <div
      className={cn(
        "fixed inset-0 z-40",
        "bg-black/50 backdrop-blur-sm",
        "transition-opacity duration-300 ease-out",
        "opacity-100",
      )}
      onClick={onClose}
      aria-hidden="true"
    />
    <dialog
      ref={dialogRef}
      className={cn(
        "relative z-50 w-full max-w-md",
        "bg-card/95 backdrop-blur-[var(--backdrop-blur)]",
        "border border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
        "rounded-[var(--radius-surface)]",
        "shadow-[var(--shadow-xl)]",
        "overflow-hidden",
        "transition-all duration-200 ease-out",
        open
          ? "opacity-100 scale-100"
          : "opacity-0 scale-95 pointer-events-none",
      )}
      role="alertdialog"
      aria-modal="true"
      aria-labelledby="message-box-title"
    />
  </div>
)

```

```

aria-describedby="message-box-description"
>
<div className={cn("p-6", bgColor, borderColor, "border-t-4")}>
  <div className="flex items-start gap-4">
    <div
      className={cn(
        "flex-shrink-0 w-10 h-10 rounded-full flex items-center
justify-center",
        bgColor,
      )}
      aria-hidden="true"
    >
      <Icon className={cn("w-5 h-5", iconColor)} strokeWidth={2.5} />
    </div>
    <div className="flex-1 min-w-0">
      <div className="flex items-start justify-between gap-4">
        <div>
          <h2
            id="message-box-title"
            className="text-lg font-semibold text-foreground leading-tight"
          >
            {title}
          </h2>
          <p
            id="message-box-description"
            className="mt-2 text-sm text-muted-foreground leading-relaxed"
          >
            {description}
          </p>
        </div>
        <span
          className={cn(
            "flex-shrink-0 px-2 py-0.5 text-xs font-medium rounded-full",
            bgColor,
            "text-foreground",
          )}
          aria-hidden="true"
        >
          {variantLabels[variant]}
        </span>
      </div>
    </div>
  </div>
  <div className="flex items-center justify-end gap-3 p-4 border-t
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]">
    <button
      type="button"
      onClick={onClose}
      className={cn(

```

```

        "inline-flex items-center justify-center gap-2",
        "h-10 px-4 py-2 text-base font-medium",
        "rounded-[var(--radius-surface)]",
        "bg-brand-primary text-white",
        "hover:bg-brand-hover",
        "active:scale-95 active:bg-brand-primary",
        "transition-all duration-150 ease-out",
        "focus-visible:outline-none",
        ringColor,
        "focus-visible:ring-2 focus-visible:ring-offset-2
focus-visible:ring-offset-background",
        "disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed",
    )}
    autoFocus
  >
    <Check className="w-4 h-4" aria-hidden="true" />
    OK
  </button>
</div>
</dialog>
</div>
);
}

```

```
export default MessageBox;
```